



OpenMP - Região Crítica

Professora: Carla Santana
carla.santana@imd.ufrn.br



Sincronização por exclusão mútua

- A comunicação assíncrona entre as threads pode gerar **condições de corrida** no programa.
- Duas ou mais threads tentam ler e escrever simultaneamente em uma mesma variável e o **resultado da computação depende da ordem de execução das threads**.
- Para que uma comunicação entre threads, via memória compartilhada, possa ocorrer de forma correta, isto é, sem gerar condições de corrida indesejáveis, **é necessário usar mecanismos de sincronização por exclusão mútua entre as threads**.
- A sincronização por exclusão mútua visa garantir que os trechos de código em cada thread, acessando variáveis compartilhadas, **não sejam executados ao mesmo tempo**. Essa restrição é necessária para evitar a inconsistência nos valores das variáveis compartilhadas



Sincronização por exclusão mútua

- Exemplo, operação que incrementa o valor de uma variável global "a" (ex., a++;).
 - Para realizar esse incremento, **três instruções de nível mais baixo (linguagem de máquina) são normalmente necessárias:**
 - ler o valor atual da variável;
 - incrementar esse valor;
 - escrever o novo valor na variável global.
- Com o uso da operação de exclusão mútua **apenas uma das threads tem acesso exclusivo à variável compartilhada**, realizando o seu incremento sem interferência da outra thread.
- **Dois processos nunca podem estar simultaneamente em suas regiões críticas.**



Sincronização por exclusão mútua

- Os mecanismos reais usados para fornecer sincronização e proteção contra condições de corrida são:
 - Operações atômicas
 - Spin Locks
 - Semáforos



Operações Atômicas

- Operações atômicas fornecem instruções que são executadas atomicamente, isto é, **sem interrupção**.
- Operadores atômicos são instruções **indivisíveis** capazes de executar a leitura e escrita de memória em uma única etapa indivisível e ininterrupta
- Operações atômicas normalmente são suportadas por mecanismos de hardware
 - Exemplo instrução TSL (Test-and-Set Lock)
 - Solução a nível de hardware
 - Impede o acesso ao barramento de memória para proibir outras CPUs de ter acesso
 - Garantir a atomicidade do mecanismo de **trava**



Spin Lock

- Para entrar em sua região crítica, um processo testa antes se há trava, verificando o valor da variável trava (uma variável compartilhada). **Se trava for 0, o processo altera essa variável para 1 e entra na região crítica.**
- A ideia por trás do *spin lock* é **executar um laço ou ficar em espera ocupada** (*busy waiting*) até que a trava (lock) seja liberada
- **A espera ociosa deveria em geral ser evitada, já que gasta tempo de CPU.**
- Um spin lock pode ser mantido por no máximo uma thread e é usado para travas mantidas por períodos curtos.
- **Instruções atômicas para garantir acesso indivisível à posição de memória correspondente à variável da trava.**



Semáforos

- Semáforos ou semáforos de contagem são travas que colocam **o processo para dormir**, limitando o número máximo de processos que possuem a trava simultaneamente.
- Por causa de sua natureza dormiente, os semáforos são adequados para travas mantidas por um longo tempo.
- A thread é bloqueada;
- **Não fica consumindo CPU enquanto espera;**
- O sistema operacional pode executar outra thread;
- **Melhor quando o tempo de espera pode ser maior e quando se quer controlar mais recursos**



The OpenMP API specification for parallel programming

[Home](#)

[Specifications](#)

[Community](#)

[Resources](#) ▾

[News & Events](#) ▾

[About](#) ▾



New OpenMP API 6.0 Examples Document

Provides programming examples of the OpenMP API, including new features found in the OpenMP 6.0 Specification.

[DOWNLOAD](#)

Latest News and Events



OpenMP ARB announces
new Board Members
van der Pas and Stotzer

Download the free
OpenMP Examples
book...



Diretiva parallel for

```
#pragma omp parallel for [clauses]  
for
```

```
#include <stdio.h>  
#include <omp.h>  
  
int main(int argc, char *argv[]) {  
    int tid;  
    # pragma omp parallel for  
    for (int i = 0; i < 12; i++)  
    {  
        tid = omp_get_thread_num();  
        printf("Alo da thread %d fazendo trabalho %d", tid, i);  
    }  
}
```

Acrescentem somente
o **#pragma omp
parallel for**
ao código do trapézio.
Ocorreu conforme
você esperava?



Criando uma região paralela

```
int main() {  
    double a = 0.0, b = 1.0;  
    int n = 1000000; // número de trapézios  
    double h = (b - a) / n;  
    double soma = 0.0;  
  
    for (int i = 1; i < n; i++) {  
        double x = a + i * h;  
        soma += f(x);  
    }  
  
    double resultado = h * ((f(a) + f(b)) / 2.0 + soma);  
  
    printf("Resultado da integral: %f\n", resultado);  
    printf("Tempo de execução: %f segundos\n", fim - inicio);  
  
    return 0;  
}
```



Diretiva critical

- Executado apenas **por uma thread de cada vez**, sendo normalmente utilizado para proteger a atualização de variáveis
- Se uma thread está em **uma seção crítica com um dado nome, nenhuma outra thread poderá estar em uma seção crítica com o mesmo nome** (embora possa haver mais de uma thread em seções críticas com nomes diferentes).



```
#pragma omp critical [( nome )]  
bloco estruturado
```



Diretiva critical

```
#pragma omp critical(secao1)
{
    // protegido por secao1
}

#pragma omp critical(secao1)
{
    // protegido por secao1
}

#pragma omp critical(secao2)
{
    // protegido por secao2
    // (independente da secao1)
}
```



Diretiva critical

```
#pragma omp critical(secao1)
{
    // p consumidor - v*
}

#pragma omp critical(secao1)
{
    // p produtor - v*
}

#pragma omp critical(secao2)
{
    // pr d*
    // (i o1)
}
```



Diretiva critical

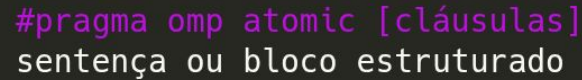
threads competem por acesso (lock)

```
#pragma omp critical(secao1)
{
    // protegido por secao1
}

#pragma omp critical(secao1)
{
    // protegido por secao1
}

#pragma omp critical(secao2)
{
    // protegido por secao2
    // (independente da secao1)
}
```

Diretiva atomic

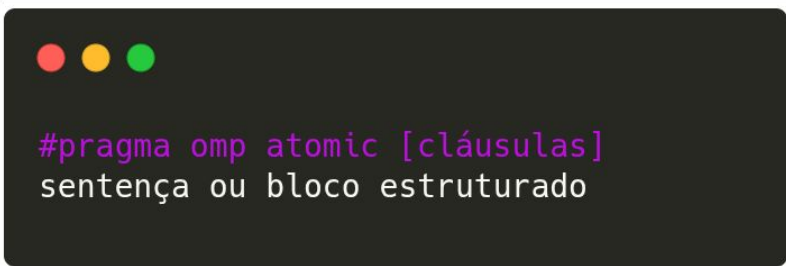


```
#pragma omp atomic [cláusulas]  
sentença ou bloco estruturado
```

- A diretiva atomic permite acessar uma variável de **modo atômico**.
- Com o uso da diretiva atomic, programas paralelos mais eficientes podem ser escritos e **com um número menor de locks**.
- Embora muito útil, a diretiva atomic não permite o seu uso com qualquer tipo de operação.

cláusula	sentença	bloco estruturado
update	x++;	
	x--;	
	++x;	
	--x;	
	x binop = expr;	
read write capture	x = x binop expr;	
	x = expr binop x;	
	v = x;	
	x = expr;	
	v = x++;	{v = x; x binop = expr;}
	v = x--;	{v = x; xOP;}
	v = ++x;	{v = x; OPx;}
	v = --x;	{x binop = expr; v = x;}
	v = x binop = expr;	{xOP; v = x;}
	v = x = x binop expr;	{OPx; v = x;}
	v = x = expr binop x;	{v = x; x = x binop expr;}
		{x = x binop expr; v = x;}
		{v = x; x = expr binop x;}
		{x = expr binop x; v = x;}
		{v = x; x = expr;}

Diretiva atomic



```
#pragma omp atomic [cláusulas]  
sentença ou bloco estruturado
```

- A diretiva atomic permite acessar uma variável de **modo atômico**.
- Com o uso da diretiva atomic, programas paralelos mais eficientes podem ser escritos e **com um número menor de locks**.
- Embora muito útil, a diretiva atomic não permite o seu uso com qualquer tipo de operação.

Sequenciando o código



Operação de Redução

- Uma redução produz **um único valor de uma variável**
- Se a variável “b” for compartilhada por todas as threads, e **for permitido o acesso apenas de uma thread por vez, não haveria paralelismo na execução do programa.**



```
b = 0;  
for (i = 0; i < n; i ++)  
    b += a[i];
```



Operação de Redução

- Uma redução produz **um único valor de uma variável**
- Se a variável “b” for compartilhada por todas as threads, e **for permitido o acesso apenas de uma thread por vez, não haveria paralelismo na execução do programa.**



```
b = 0;  
for (i = 0; i < n; i ++)  
    b += a[i];
```

Podemos fazer
uma cópia de **b**
para cada thread
e depois somar
apenas as cópias

Cláusula reduction



```
reduction(operador:lista-de-variáveis)
```

- Cada thread pode acumula uma soma parcial em sua própria cópia privada
- As cópias são reduzidas para se obter o resultado na thread master.



```
b = 0;  
# pragma omp parallel for reduction (+: b)  
for (i = 0; i < n; i ++)  
    b += a[i];
```

Tarefa 6



Implemente diferentes versões de um programa para o cálculo de integrais definidas utilizando o **método do trapézio**. O programa deve permitir calcular numericamente a integral de uma função ($f(x)$) em um intervalo $[a,b]$, utilizando (n) subdivisões.

Devem ser implementadas as seguintes versões:

- **Sequencial**
- **Paralela sem tratamento para região crítica**
- **Paralela utilizando `critical`**
- **Paralela utilizando `atomic`**
- **Paralela utilizando `reduction`**

Após a implementação, compare os resultados obtidos e os **tempos médios de execução** de cada versão à medida que o número de subdivisões (n) aumenta, utilizando **gráficos**, para apresentar e analisar os resultados.

Para cada experimento, discuta:

1. A corretude dos resultados obtidos;
2. As diferenças entre os resultados das versões sequencial e paralelas;
3. O impacto da ausência de sincronização sobre o resultado;
4. As diferenças de desempenho entre as abordagens utilizando `critical`, `atomic` e `reduction`;
5. Como o aumento do número de subdivisões (n) influencia o tempo de execução e o desempenho das diferentes versões.

Ao final, explique e justifique as diferenças observadas nos resultados e nos tempos de execução de cada implementação.